

Pseudocodice di DemoAlgorithm

Implementazione MaxEnt IRL da dimostrazioni

1 Notazione

Una transizione è $x = (o, a, r^{\text{true}}, s', d)$, dove o è l'osservazione, a l'azione, s' il vettore di stato successivo e d il flag terminale. Una traiettoria è $\tau = (x_1, \dots, x_{|\tau|})$. Per il membro k dell'ensemble di reward si definisce

$$R_{\theta_k}(\tau) = \sum_{x_t \in \tau} r_{\theta_k}(o_t, a_t, s'_t, d_t).$$

Le traiettorie esperte sono indicate con $\mathcal{T}^E$, quelle campionate dalla policy corrente con $\mathcal{T}^M$, la policy con π_ϕ e la temperatura con T . È importante distinguere due scale del reward:

$$\begin{aligned} r_\theta^{\text{raw}}(x) &= r_\theta(x), & \text{usato nelle loss e nei gradienti del reward model,} \\ r_\theta^{\text{agent}}(x) &= \frac{r_\theta^{\text{raw}}(x) - \mu}{\sigma + 10^{-8}}, & \text{usato dall'agente e dalle predizioni } \texttt{predict}. \end{aligned}$$

Per `maxent_corrected`, una traiettoria può essere divisa in frammenti consecutivi f di lunghezza massima L_f ; se `fragment_length=None`, ogni traiettoria costituisce un unico frammento. Si indicano con $R_\theta(f)$ e $\log q(f) = \sum_{t \in f} \log \pi_\phi(a_t \mid o_t)$ rispettivamente il return appreso e la log-probabilità del frammento.

2 Costruzione e ciclo principale

Algorithm 1 Inizializzazione di DEMOALGORITHM

Input: ambiente $\mathcal{E}$, agente π_ϕ , dimostrazioni $\mathcal{T}^E$, tipo di loss

Input: iperparametri del reward, rollout e normalizzazione

Output: algoritmo inizializzato

```
1: if  $\mathcal{T}^E = \emptyset$  o tipo di loss non valido then
2:   solleva errore
3: end if
4: if loss = maxent_corrected e non esiste un ambiente di rollout dedicato then
5:   solleva errore
6: end if
7: if  $T \leq 0$ , passi gradiente  $\leq 0$  o batch size  $\leq 0$  then
8:   solleva errore
9: end if
10: if fragment_length non è None né un intero positivo then
11:   solleva errore
12: end if
13: inizializza la classe base, salva configurazione e dataset di debug
14:  $r_\theta \leftarrow \text{MAKEREWARDENSEMBLE}(\mathcal{E})$ 
15: configura il logger dell'agente
16: crea il generatore di traiettorie con agente, reward appreso ed esplorazione
17: if l'agente ha un replay buffer then
18:   if il buffer supporta il reward relabeling then
19:     collega  $r_\theta$  al buffer e configura il relabeling
20:   else if il relabeling è richiesto then
21:     solleva errore
22:   end if
23: end if
24: for all membri  $r_{\theta_k}$  dell'ensemble do
25:   crea Adam( $\theta_k, \eta_r$ , weight decay =  $\lambda$ )
26: end for
```

Algorithm 2 TRAIN: ciclo alternato reward-policy

Input: timesteps totali H , timesteps per iterazione h , intervalli di log, diagnostica, scatter e checkpoint

Output: agente addestrato π_ϕ

```
1:  $N \leftarrow \lfloor H/h \rfloor$ 
2: if  $h_0 > 0$  then  $\triangleright$  bootstrap iniziale
3:    $\mathcal{T}^M \leftarrow \text{SAMPLEROLLOUT}(h_0, 0)$ 
4:    $\text{TRAINREWARDMODEL}()$ 
5:    $\text{UPDATEAGENTREWARDNORMALIZATION}()$ 
6:    $\text{TRAINAGENT}(h_0)$ 
7: end if
8: for  $i = 0, \dots, N - 1$  do
9:    $h_\varepsilon \leftarrow \lfloor \text{exploration\_frac} \cdot h \rfloor$ 
10:   $\mathcal{T}^M \leftarrow \text{SAMPLEROLLOUT}(h, h_\varepsilon)$ 
11:  if la diagnostica d'imitazione è prevista a  $i$  then
12:     $\text{LOGIMITATIONDIAGNOSTICS}()$ 
13:  end if
14:   $X \leftarrow$  tutte le transizioni contenute in  $\mathcal{T}^M$ 
15:   $\text{LOGVALIDATIONSNAPSHOT}(X, \text{pre\_update})$ 
16:   $\text{TRAINREWARDMODEL}()$ 
17:   $\text{UPDATEAGENTREWARDNORMALIZATION}()$ 
18:   $\text{LOGVALIDATIONSNAPSHOT}(X, \text{post\_update})$ 
19:   $\text{LOGOUTCOMEReturns}()$ 
20:   $\text{LOGREPLAYREWARDStaleness}()$ 
21:  if esiste il dataset di debug e lo scatter è previsto a  $i$  then
22:     $\text{LOGRETURNSCATTER}(\mathcal{T}^{\text{debug}}, \text{post\_update}, i)$ 
23:  end if
24:   $\text{TRAINAGENT}(h)$ 
25:  registra iterazione, timesteps e tempi; svuota il logger
26:  if è previsto un checkpoint then
27:     $\text{SAVECHECKPOINT}(i + 1)$ 
28:  end if
29: end for
30: return  $\pi_\phi$ 
```

3 Campionamento e funzioni di perdita

Algorithm 3 Campionamento di traiettorie e return

```
1: function  $\text{SAMPLETRAJECTORIES}$ 
2:   campiona senza rimpiazzo  $\widehat{\mathcal{T}}^E \subseteq \mathcal{T}^E$  di cardinalità  $\min(B_E, |\mathcal{T}^E|)$ 
3:   campiona senza rimpiazzo  $\widehat{\mathcal{T}}^M \subseteq \mathcal{T}^M$  di cardinalità  $\min(B_M, |\mathcal{T}^M|)$ 
4:   return  $(\widehat{\mathcal{T}}^E, \widehat{\mathcal{T}}^M)$ 
5: end function
6: function  $\text{SAMPLERETURNS}(r_{\theta_k})$ 
7:    $(\widehat{\mathcal{T}}^E, \widehat{\mathcal{T}}^M) \leftarrow \text{SAMPLETRAJECTORIES}()$ 
8:    $\mathbf{R}^E \leftarrow [R_{\theta_k}(\tau) : \tau \in \widehat{\mathcal{T}}^E]$ 
9:    $\mathbf{R}^M \leftarrow [R_{\theta_k}(\tau) : \tau \in \widehat{\mathcal{T}}^M]$ 
10:  return  $(\mathbf{R}^E, \mathbf{R}^M, \widehat{\mathcal{T}}^E, \widehat{\mathcal{T}}^M)$ 
11: end function
```

Algorithm 4 REWARDLOSS

Input: membro r_{θ_k} e tipo di loss ℓ

Output: loss scalare differenziabile $\mathcal{L}$

```
1: if  $\ell = \text{maxent\_corrected}$  then
2:    $(\widehat{\mathcal{T}}^E, \widehat{\mathcal{T}}^M) \leftarrow \text{SAMPLETRAJECTORIES}()$ 
3:    $\mathbf{R}_F^E \leftarrow \text{FRAGMENTRETURNS}(r_{\theta_k}, \widehat{\mathcal{T}}^E)$ 
4:    $\mathbf{R}_F^M \leftarrow \text{FRAGMENTRETURNS}(r_{\theta_k}, \widehat{\mathcal{T}}^M)$ 
5:    $\log \mathbf{q}_F \leftarrow \text{FRAGMENTLOGPROBS}(\widehat{\mathcal{T}}^M)$ 
6:    $\mathbf{z} \leftarrow \mathbf{R}_F^M / T - \log \mathbf{q}_F$ 
7:    $\text{RECORDMAXENTCORRECTEDSTEP}(\mathbf{R}_F^M / T, \log \mathbf{q}_F, \mathbf{z})$ 
8:   return  $-\text{mean}(\mathbf{R}_F^E) / T + \text{logsumexp}(\mathbf{z}) - \log |\mathbf{z}|$ 
9: end if
10:  $(\mathbf{R}^E, \mathbf{R}^M, \widehat{\mathcal{T}}^E, \widehat{\mathcal{T}}^M) \leftarrow \text{SAMPLERETURNS}(r_{\theta_k})$ 
11:  $L_E \leftarrow -\text{mean}(\mathbf{R}^E)$ 
12: if  $\ell \in \{\text{demo}, \text{demo\_loss}\}$  then
13:   return  $L_E + \text{mean}(\mathbf{R}^M)$ 
14: else if  $\ell = \text{demo\_corrected}$  then
15:    $\mathbf{m} \leftarrow \text{DEMOCORRECTEDMARGINS}(\mathbf{R}^E, \mathbf{R}^M, \widehat{\mathcal{T}}^E, \widehat{\mathcal{T}}^M)$ 
16:   return  $\text{mean}_i[\text{softplus}(-m_i / T)]$ 
17: else if  $\ell = \text{maxent\_2}$  then
18:    $\mathbf{R} \leftarrow \mathbf{R}^M \parallel \mathbf{R}^E$ 
19:   return  $L_E + \text{logsumexp}(\mathbf{R}) - \log |\mathbf{R}|$ 
20: else if  $\ell = \text{maxent}$  then
21:   return  $L_E + \text{logsumexp}(\mathbf{R}^M) - \log |\mathbf{R}^M|$ 
22: end if
```

Algorithm 5 Funzioni ausiliarie per return e probabilità della policy

```
1: function DEMOCORRECTEDMARGINS( $\mathbf{R}^E, \mathbf{R}^M, \widehat{\mathcal{T}}^E, \widehat{\mathcal{T}}^M$ )
2:    $n \leftarrow \min(|\mathbf{R}^E|, |\mathbf{R}^M|)$ 
3:   return  $[R_i^E / |\widehat{\tau}_i^E| - R_i^M / |\widehat{\tau}_i^M|]_{i=1}^n$ 
4: end function
5: function TRAJECTORYLOGPOLICYPROB( $\tau$ )
6:   if ogni transizione contiene  $\log \pi_\phi(a_t | o_t)$  then
7:     return  $\sum_{t \in \tau} \log \pi_\phi(a_t | o_t)$  memorizzato
8:   else
9:     calcola in batch  $\log \pi_\phi(a_t | o_t)$  tramite l'agente
10:    return la somma delle log-probabilità
11:   end if
12: end function
13: function TRAJECTORYSUMREWARD( $r_{\theta_k}, \tau$ )
14:   converti osservazioni, azioni, stati successivi e flag terminali in tensori
15:   return  $\sum_{t \in \tau} r_{\theta_k}(o_t, a_t, s'_t, d_t)$  mantenendo il grafo dei gradienti
16: end function
17: function FRAGMENTRETURNS( $r_{\theta_k}, \mathcal{T}$ )
18:   dividi ogni traiettoria in blocchi consecutivi di lunghezza  $L_f$ 
19:   return il vettore delle somme di reward per blocco, mantenendo i gradienti
20: end function
21: function FRAGMENTLOGPROBS( $\mathcal{T}$ )
22:   usa la stessa divisione e somma le log-probabilità per ogni blocco
23:   return il vettore allineato ai return dei frammenti
24: end function
```

In forma compatta, le loss implementate sono

$$\begin{aligned}
\mathcal{L}_{\text{demo}} &= -\overline{R^E} + \overline{R^M}, \\
\mathcal{L}_{\text{maxent}} &= -\overline{R^E} + \text{LSE}(\mathbf{R}^M) - \log n_M, \\
\mathcal{L}_{\text{maxent2}} &= -\overline{R^E} + \text{LSE}(\mathbf{R}^M \parallel \mathbf{R}^E) - \log(n_M + n_E), \\
\mathcal{L}_{\text{demo_corr}} &= \text{mean}_i \text{softplus} \left(-\frac{R_i^E / |\tau_i^E| - R_i^M / |\tau_i^M|}{T} \right), \\
\mathcal{L}_{\text{maxent_corr}} &= -\overline{R_F^E} / T + \text{LSE}(\mathbf{R}_F^M / T - \log \mathbf{q}_F) - \log n_F.
\end{aligned}$$

Con `fragment_length=None`, $n_F = n_M$ e la formula coincide con quella a traiettoria intera. Con frammenti più corti l'obiettivo diventa una versione MaxEnt locale e sperimentale: i frammenti consecutivi non sono campioni i.i.d., quindi non conservano automaticamente la consistenza dell'importance sampling a traiettoria intera.

4 Ottimizzazione, normalizzazione e rollout

Algorithm 6 TRAINREWARDMODEL

```

1: if  $\mathcal{T}^M = \emptyset$  then
2:   return
3: end if
4: for all  $(r_{\theta_k}, \text{Adam}_k)$  dell'ensemble do  $\triangleright$  esecuzione tramite executor a un worker
5:   imposta  $r_{\theta_k}$  in modalità training
6:   for  $g = 1, \dots, G$  do
7:      $L \leftarrow \text{REWARDLOSS}(r_{\theta_k})$ ; verifica che  $L$  sia finita
8:     azzera i gradienti; esegui backpropagation
9:      $\|\nabla \theta_k\|_2 \leftarrow \sqrt{\sum_p \|\nabla p\|_2^2}$ ; verifica che sia finita
10:    esegui un passo di Adam
11:   end for
12: end for
13: LOGREWARDLOSSDIAGNOSTICS()
14: LOGMAXENTCORRECTEDSTEPPDIAGNOSTICS()
15: registra media/massimo delle norme dei gradienti,  $\sqrt{\sum_p \|p\|_2^2}$  e tempi

```

Algorithm 7 Helper dell'ottimizzazione e logging d'iterazione

```

1: function TRAINMEMBER( $r_{\theta_k}, \text{Adam}_k$ )
2:   esegui i  $G$  passi loss-backpropagation-Adam descritti in TRAINREWARDMODEL
3:   return la lista delle norme dei gradienti
4: end function
5: function GRADNORM( $M$ )
6:   return 0 se nessun parametro ha gradiente, altrimenti  $\sqrt{\sum_{p: \nabla p \neq \emptyset} \|\nabla p\|_2^2}$ 
7: end function
8: function PARAMNORM( $M$ )
9:   return  $\sqrt{\sum_{p \in M} \|p\|_2^2}$  usando parametri staccati dal grafo
10: end function
11: function LOGITERATION( $t_0, i$ )
12:   registra  $i$ , i timesteps totali dell'agente, il tempo trascorso e il tempo di logging
13:   esegui il dump del logger
14: end function

```

Algorithm 8 UPDATEAGENTREWARDNORMALIZATION

```
1: if normalizzazione disabilitata o  $\mathcal{T}^M = \emptyset$  then
2:   return
3: end if
4:  $X \leftarrow$  tutte le transizioni di  $\mathcal{T}^M$ ; se  $X = \emptyset$ , return
5:  $\mathbf{r} \leftarrow r_{\theta}^{\text{unnormalized}}(X)$ ; verifica finitezza
6:  $\mu \leftarrow \text{mean}(\mathbf{r})$ ,  $\sigma \leftarrow \text{std}(\mathbf{r})$ 
7: se  $\sigma \leq 10^{-8}$  poni  $\sigma \leftarrow 1$ 
8:  $\mu_{\text{new}} \leftarrow (1 - \alpha)\mu_{\text{old}} + \alpha\mu$ 
9:  $\sigma_{\text{new}} \leftarrow (1 - \alpha)\sigma_{\text{old}} + \alpha\sigma$ 
10: imposta  $\mu_{\text{new}}$  e  $\max(\sigma_{\text{new}}, 10^{-8})$  nel reward model
11: calcola i reward normalizzati, verifica finitezza e registra le statistiche
```

Algorithm 9 Rollout, score e addestramento dell'agente

```
1: function SAMPLEROLLOUT( $h, h_{\varepsilon}$ )
2:    $\mathcal{T}^M \leftarrow \text{trajectory\_generator.sample}(h, h_{\varepsilon})$ 
3:   calcola per ogni  $\tau$ : return vero, SCORETRAJECTORY( $\tau$ ) e lunghezza
4:   registra medie, tempi e LOGACTIONBOUNDARIES( $\mathcal{T}^M$ )
5:   return  $\mathcal{T}^M$ 
6: end function
7: function SCORETRAJECTORY( $\tau$ )
8:   return  $\sum_{t \in \tau} r_{\theta}^{\text{predict}}(o_t, a_t, s'_t, d_t)$ 
9: end function
10: function TRAINAGENT( $h, \logInterval$ )
11:   esegui trajectory_generator.train per  $h$  passi e registra il tempo
12: end function
13: function LOGACTIONBOUNDARIES( $\mathcal{T}^M$ )
14:   if lo spazio delle azioni non è Box then
15:     return
16:   end if
17:   concatena tutte le azioni e confrontale con i limiti inferiori/superiori ( $\text{atol} = 10^{-6}$ )
18:   registra frazione di azioni con almeno una componente al limite e frazione di componenti al
   limite
19: end function
```

5 Diagnostica d'imitazione

Algorithm 10 AUC del classificatore esperto-agente

```

1: function LOGIMITATIONDIAGNOSTICS
2:    $A \leftarrow \text{STATEACTIONCLASSIFIERAUC}(\mathcal{T}^E, \mathcal{T}^M)$ 
3:   se  $A$  è definita, registrala insieme al tempo impiegato
4: end function
5: function STATEACTIONCLASSIFIERAUC( $\mathcal{T}^E, \mathcal{T}^M$ )
6:   dividi le traiettorie non vuote esperte e agente in train/validation 80/20
7:   if uno split non è possibile then
8:     return NONE
9:   end if
10:  estrai feature  $[o; a]$  da ciascuno dei quattro insiemi
11:  if un insieme è vuoto o le dimensioni non coincidono then
12:    return NONE
13:  end if
14:  bilancia le due classi in train e validation fino a 5000 esempi per classe
15:  costruisci etichette  $y = 1$  per esperto e  $y = 0$  per agente
16:  standardizza train e validation con media/deviazione del train
17:  inizializza  $w = 0, b = 0$ 
18:  for  $j = 1, \dots, 100$  do
19:     $z \leftarrow \text{clip}(Xw + b, -30, 30); p \leftarrow \sigma(z)$ 
20:     $\nabla_w \leftarrow X^\top(p - y)/|X| + 10^{-4}w; \nabla_b \leftarrow \text{mean}(p - y)$ 
21:     $w \leftarrow w - 0.1\nabla_w; b \leftarrow b - 0.1\nabla_b$ 
22:  end for
23:  return BINARYAUC( $y_{val}, X_{val}w + b$ )
24: end function

```

Algorithm 11 Helper della diagnostica d'imitazione

```

1: function TRAJECTORYTRAINVALIDATIONSPLIT( $\mathcal{T}, seed$ )
2:   elimina traiettorie vuote; se ne restano meno di due, return NONE
3:   mescola gli indici con  $seed$  e assegna a validation  $\min(\max(1, \text{round}(0.2|\mathcal{T}|)), |\mathcal{T}| - 1)$ 
   traiettorie
4:   return (train, validation)
5: end function
6: function STATEACTIONFEATURES( $\mathcal{T}$ )
7:   per ogni transizione valida crea la riga finita  $[\text{vec}(o); \text{vec}(a)]$ 
8:   return lo stack delle righe, oppure una matrice vuota se non impilabili
9: end function
10: function BALANCEFEATURECLASSES( $X^+, X^-$ )
11:    $n \leftarrow \min(|X^+|, |X^-|, 5000)$ 
12:   seleziona  $n$  indici equispaziati da ciascuna classe; return i due sottoinsiemi
13: end function
14: function BINARYAUC( $y, s$ )
15:    $n_+ \leftarrow \sum[y = 1], n_- \leftarrow \sum[y = 0]$ ; se uno è zero, return NONE
16:   assegna ai punteggi ranghi medi  $r_i$ 
17:   return  $\frac{\sum_{i:y_i=1} r_i - n_+(n_+ + 1)/2}{n_+n_-}$ 
18: end function

```

6 Diagnostica del reward

Algorithm 12 Return diagnostici e sottoinsieme equispaziato

```

1: function EVENSUBSET( $A, m$ )
2:   sostituisci  $m \leq 0$  con 1; se  $|A| \leq m$ , return  $A$ 
3:   return gli elementi in  $m$  posizioni equispaziate fra 0 e  $|A| - 1$ 
4: end function
5: function DIAGNOSTICRETURNS( $r$ )
6:    $\widehat{\mathcal{T}}^E \leftarrow \text{EVENSUBSET}(\mathcal{T}^E, B_E)$ ;  $\widehat{\mathcal{T}}^M \leftarrow \text{EVENSUBSET}(\mathcal{T}^M, B_M)$ 
7:   calcola i vettori dei return con TRAJECTORYSUMREWARD
8:   return return e traiettorie selezionate
9: end function

```

Algorithm 13 LOGREWARDLOSSDIAGNOSTICS

```

1: if  $\mathcal{T}^M = \emptyset$  then
2:   return
3: end if
4: imposta il modello in evaluation e disabilita i gradienti
5: ottieni  $\mathbf{R}^E, \mathbf{R}^M$  diagnostici; verifica che siano finiti
6: registra medie, margine, deviazione, media assoluta, minimo e massimo
7: if loss = demo o demo_loss then
8:   registra  $-\overline{R}^E + \overline{R}^M$ , margine e scala
9: else if loss = demo_corrected then
10:  calcola i margini medi per passo e registra softplus loss, margine e scala
11: else if loss = maxent_2 then
12:   $z \leftarrow \mathbf{R}^M \parallel \mathbf{R}^E$ ;  $w \leftarrow \text{softmax}(z)$ 
13:  registra loss, partizione, masse esperto/modello, peso top-1 ed  $\text{ESS} = 1 / \sum_i w_i^2$ 
14:  registra  $\text{ESS}/|w|$  e genera un warning se è minore di 0.1
15: else
16:   $z \leftarrow \mathbf{R}^M$ , oppure  $\mathbf{R}_F^M / T - \log \mathbf{q}_F$  per maxent_corrected
17:   $w \leftarrow \text{softmax}(z)$ ;  $\text{ESS} \leftarrow 1 / \sum_i w_i^2$ 
18:  registra loss, partizione, massa top-1/top-5, ESS ed eventualmente media di  $\log q$ 
19:  registra  $\text{ESS}/|w|$  e genera un warning se è minore di 0.1
20: end if
21: ripristina la modalità training

```

Algorithm 14 LOGESSFRACTION

Input: prefisso di log, ESS, numero di elementi n

```

1:  $f \leftarrow \text{ESS}/n$ ; registra  $f$ 
2: if  $f < 0.1$  then
3:   genera un warning per la degenerazione dei pesi
4: end if

```

Algorithm 15 Diagnostica fedele ai gradient step di `maxent_corrected`

```
1: function RECORDMAXENTCORRECTEDSTEP( $\mathbf{u}$ ,  $\log \mathbf{q}$ ,  $\mathbf{z}$ )
2:    $\mathbf{w} \leftarrow \text{softmax}(\mathbf{z})$ , dove  $\mathbf{u} = \mathbf{R}_F^M/T$  e  $\mathbf{z} = \mathbf{u} - \log \mathbf{q}$ 
3:   calcola ESS, ESS fraction, numero di frammenti, massa top-1 e top-5
4:   calcola  $\text{Var}(\mathbf{z})$ ,  $\text{Var}(\mathbf{u})$ ,  $\text{Var}(\log \mathbf{q})$  e  $\text{Cov}(\mathbf{u}, \log \mathbf{q})$ 
5:   conserva i valori relativi al batch che produrrà realmente il gradiente
6: end function
7: function LOGMAXENTCORRECTEDSTEPIAGNOSTICS
8:   if non esistono record then
9:     return
10:  end if
11:  registra la media di ogni quantità sui gradient step e membri dell'ensemble
12:  registra inoltre ESS minima/mediana, ESS fraction minima/mediana, top-1 massima e
    varianza dei logits massima
13: end function
```

La decomposizione usata per diagnosticare la degenerazione dei pesi è

$$\text{Var}(\mathbf{z}) = \text{Var}(\mathbf{R}_F^M/T) + \text{Var}(\log \mathbf{q}_F) - 2 \text{Cov}(\mathbf{R}_F^M/T, \log \mathbf{q}_F).$$

Essa permette di capire se la dispersione dei logits, e quindi il collasso dell'ESS, deriva dalla scala del reward, dalla probabilità della proposal policy o dalla loro covarianza.

Algorithm 16 Staleness del reward nel replay buffer

```
1: function LOGREPLAYREWARDSTALENESS( $B = 2048$ )
2:   if il replay buffer non supporta il campionamento della staleness then
3:     return
4:   end if
5:   campiona reward memorizzati  $r^{old}$  e correnti  $r^{new}$ ; se assenti, return
6:    $\Delta \leftarrow r^{new} - r^{old}$ ,  $\sigma_{new} \leftarrow \text{std}(r^{new})$ ;  $d \leftarrow \sigma_{new}$  se  $\sigma_{new} > 10^{-8}$ , altrimenti  $d \leftarrow 1$ 
7:   registra numerosità, medie/std, statistiche di  $\Delta$ , percentile 95 di  $|\Delta|$ 
8:   registra  $\text{mean } |\Delta|/d$ , frazione di cambi di segno e stato del relabeling
9:   if entrambe le deviazioni standard sono  $> 10^{-8}$  then
10:    registra  $\text{corr}(r^{old}, r^{new})$ 
11:   end if
12: end function
```

Algorithm 17 Return del reward corrente raggruppati per esito

```
1: function LOGOUTCOMERETURNS
2:    $\gamma \leftarrow$  discount factor dell'agente
3:   for all traiettorie non vuote con transizione terminale e stato one-hot valido do
4:     classifica l'esito in arrived, collided, offroad, timeout, teleported o removed unknown
5:     calcola i reward per step raw  $\mathbf{r}^{raw}$  e normalizzati  $\mathbf{r}^{norm}$ 
6:     calcola  $R^{raw} = \sum_t r_t^{raw}$ ,  $R^{norm} = \sum_t r_t^{norm}$  e  $R^{disc} = \sum_t \gamma^t r_t^{norm}$ 
7:     conserva reward medio per step, reward terminale e lunghezza
8:   end for
9:   per ogni esito registra sempre il conteggio e, se non vuoto, la media delle altre quantità
10: end function
```

R^{disc} è un proxy del return ottimizzato da SAC: è particolarmente fedele quando il replay buffer effettua il relabeling, ma non comprende il termine di entropia. Per i timeout è un proxy parziale, perché SAC può effettuare bootstrap oltre la troncatura mentre la somma si ferma all'ultimo step osservato.

Algorithm 18 Validazione puntuale del reward

```
1: function LOGVALIDATIONSNAPSHOT( $X, stage$ )
2:   LOGREWARDVALIDATION( $X, current\_rollout/stage$ )
3:   LOGRETURNRANKING( $\mathcal{T}^M, current\_rollout/stage$ )
4:   if esiste un dataset di debug then
5:     ripeti le due operazioni sul dataset e sulle sue traiettorie
6:   end if
7: end function
8: function RUNREWARDINFERENCE( $X$ )
9:   imposta evaluation e disabilita gradienti
10:  estrai reward veri, osservazioni, azioni, stati e flag terminali
11:   $\hat{r} \leftarrow r_{\theta}.\text{predict}(X)$ 
12:   $u \leftarrow \text{std}_k(r_{\theta_k}(X))$ ; verifica che  $\hat{r}, u$  siano finiti
13:  ripristina training; return ( $r^{true}, \hat{r}, u, s'$ )
14: end function
15: function LOGREWARDVALIDATION( $X, prefix$ )
16:  ottieni  $(\cdot, \hat{r}, u, s') \leftarrow \text{RUNREWARDINFERENCE}(X)$ 
17:  costruisci maschere per arrived, collided, offroad, timeout e running
18:  registra statistiche globali e medie per stato mediante RECORDMASKEDMEAN
19:  registra gap arrived-collided e arrived-running quando definiti
20:  registra incertezza media dell'ensemble globale e durante running
21: end function
22: function RECORDMASKEDMEAN( $key, v, m$ )
23:   if esiste almeno un elemento selezionato da  $m$  then
24:     registra e return mean( $v[m]$ )
25:   end if
26:   return NONE
27: end function
```

Algorithm 19 Traiettorie di debug e correlazione dei return

```
1: function SPLITINTOTRAJECTORIES( $X$ )
2:   inizializza lista  $L$  e traiettoria corrente  $\tau$ 
3:   for all  $x \in X$  do
4:     aggiungi  $x$  a  $\tau$ 
5:     if  $x.done$  then
6:       aggiungi  $\tau$  a  $L$  e crea una nuova traiettoria vuota
7:     end if
8:   end for
9:   se la traiettoria finale non è vuota, aggiungila a  $L$ ; return  $L$ 
10: end function
11: function LOGRETURNRANKING( $\mathcal{T}, prefix$ )
12:   if  $|\mathcal{T}| < 2$  then
13:     return
14:   end if
15:   for all  $\tau \in \mathcal{T}$  do
16:      $R^{true}(\tau) \leftarrow \sum_{t \in \tau} r_t^{true}$ 
17:      $\hat{R}(\tau) \leftarrow \sum_{t \in \tau} \text{RUNREWARDINFERENCE}(t). \hat{r}$ 
18:   end for
19:    $\rho \leftarrow \text{Spearman}(\mathbf{R}^{true}, \hat{\mathbf{R}})$ 
20:   registra se  $\rho$  è definita e, in tal caso, il suo valore
21: end function
22: function LOGRETURNSCATTER( $\mathcal{T}, prefix, i$ )
23:   if W&B non è attivo o  $|\mathcal{T}| < 2$  then
24:     return
25:   end if
26:   calcola per ogni traiettoria  $(\hat{R}(\tau), R^{true}(\tau))$ 
27:   crea un'immagine scatter con predetto sull'asse  $x$  e vero sull'asse  $y$ 
28:   usa limiti verticali fissi derivati dai return veri e registra l'immagine su W&B
29: end function
```

7 Metriche registrate: calcolo e interpretazione

Tutte le metriche personalizzate, eccetto quelle con prefisso `agent/`, usano `iterations` come asse semantico in W&B. Le metriche `agent/` usano invece `agent/time/total_timesteps`. Molte metriche secondarie sono nascoste dal workspace automatico, ma restano disponibili nella history e possono essere aggiunte manualmente ai pannelli.

7.1 Rollout e comportamento della policy

Metrica	Calcolo e utilità
<code>rollout/mean_true_reward</code>	Media di $\sum_t r_t^{true}$ sulle traiettorie appena raccolte; misura la prestazione ambientale reale.
<code>rollout/mean_model_reward</code>	Media di $\sum_t r_t^{agent}$; mostra come il reward corrente valuta la policy.
<code>rollout/mean_length</code>	Lunghezza media delle traiettorie; aiuta a rilevare terminazioni precoci o timeout.
<code>rollout/action_at_bound_fraction</code>	Frazione di azioni con almeno una componente entro 10^{-6} da un limite dello spazio Box; segnala saturazione.
<code>rollout/action_component_at_bound_fraction</code>	Frazione di singole componenti d'azione al limite.

Metrica	Calcolo e utilità
imitation/state_action_auc	AUC su validation di un classificatore logistico lineare che distingue feature $[o; a]$ esperte ($y = 1$) da agent ($y = 0$). Un valore vicino a 0.5 indica occupazioni difficili da distinguere; verso 1 indica forte separazione.

Le metriche SB3 con prefisso **agent/** dipendono dall'algoritmo. Nel workspace sono rese visibili in particolare successi, collisioni, off-road, return ambientale, critic loss e coefficiente di entropia; servono a valutare prestazione e stabilità di SAC indipendentemente dal reward model.

7.2 Reward model, loss e importance sampling

Metrica	Calcolo e utilità
reward/expert_return_mean	$\overline{R^E}$ sul sottoinsieme diagnostico; dovrebbe risultare maggiore del return agent se il reward discrimina correttamente.
reward/model_return_mean	$\overline{R^M}$ sul sottoinsieme diagnostico.
reward/expert_model_margin	$\overline{R^E} - \overline{R^M}$; margine positivo favorisce le dimostrazioni.
reward/return_std, return_abs_mean, return_min, return_max	Dispersione, scala assoluta ed estremi dei return esperti e agent concatenati; rilevano deriva o esplosione della scala.
reward/loss	Loss ricalcolata senza gradienti su un sottoinsieme diagnostico equispaziato; è confrontabile nel tempo ma non coincide necessariamente con l'ultimo minibatch di training.
reward/grad_norm, grad_norm_max	Media e massimo di $\sqrt{\sum_p \ \nabla p\ _2^2}$ sui gradient step; diagnosticano gradienti nulli o esplosivi.
reward/weight_norm	$\sqrt{\sum_p \ p\ _2^2}$ sull'intero ensemble; misura la crescita dei parametri.
reward/normalization_*	Media/std raw del rollout, valori applicati dopo smoothing con α e media/std in uscita; verificano la scala fornita a SAC.
reward/demo_margin, demo_scale_*	Margine esperto-agent e scala per la loss demo storica.
reward/demo_corrected_margin, demo_corrected_scale_std	Margine medio per step e scala per la ranking loss corretta.
reward/maxent*_partition*	$LSE(\mathbf{z}) - \log n$; stima il termine di partizione della loss selezionata.
reward/maxent*_top1_softmax_weight	$\max_i w_i$; vicino a 1 indica che un solo campione domina la partizione.
reward/maxent*_top5_softmax_mass	Somma dei cinque pesi maggiori; misura la concentrazione complessiva.
reward/maxent*_effective_sample_size	$1/\sum_i w_i^2$; numero effettivo di traiettorie o frammenti che contribuiscono.
reward/maxent*_effective_sample_fraction	ESS/n ; permette confronti tra batch di dimensione diversa. Sotto 0.1 viene emesso un warning.
reward/maxent2_expert_softmax_mass, model_softmax_mass	Massa totale assegnata rispettivamente a esempi esperti e agent nella partizione congiunta.
reward/maxent2_top1_is_expert	Indicatore 0/1 che segnala se il campione col peso massimo è esperto.
reward/maxent_corrected_log_q_mean	Media delle log-probabilità della proposal policy; valori molto negativi crescono tipicamente con l'orizzonte.

Il simbolo **reward/maxent*** nella tabella rappresenta i prefissi **maxent**, **maxent2** o **maxent_corrected** pertinenti alla loss configurata; i nomi concreti sono quelli costruiti dal relativo prefisso nel codice.

Le metriche `reward/maxent_corrected_grad/*` sono calcolate sui minibatch che hanno prodotto realmente i gradienti. Per `ess`, `ess_fraction`, `n_fragments`, `top1_softmax_weight`, `top5_softmax_mass`, `logit_var`, `var_scaled_return`, `var_log_q` e `cov_return_log_q` viene registrata la media. Sono inoltre registrati `ess_min`, `ess_median`, `ess_fraction_min`, `ess_fraction_median`, `top1_softmax_weight_max` e `logit_var_max`, perché una media può nascondere singoli aggiornamenti degeneri.

7.3 Validazione, generalizzazione e replay buffer

I prefissi di validazione sono `reward_val/current_rollout/(pre_update|post_update)` e `reward_val/debug_dataset/(pre_update|post_update)`. Il rollout corrente misura il comportamento on-policy recente; il dataset di debug è fisso e permette di distinguere miglioramento da overfitting o deriva.

Suffisso/metrica	Calcolo e utilità
<code>reward_mean</code> , <code>reward_std</code> , <code>reward_min</code> , <code>reward_max</code>	Statistiche dei reward normalizzati per transizione.
<code>reward_running</code> , <code>reward_arrived</code> , <code>reward_collided</code> , <code>reward_offroad</code> , <code>reward_timeout</code>	Media condizionata allo stato successivo; mostra quali eventi il reward incentiva o penalizza.
<code>gap_arrived_collided</code>	$\bar{r}_{arrived} - \bar{r}_{collided}$; dovrebbe essere positivo.
<code>gap_arrived_running</code>	$\bar{r}_{arrived} - \bar{r}_{running}$; verifica il bonus terminale rispetto alla guida ordinaria.
<code>ensemble_std</code> , <code>ensemble_std_running</code>	Media della deviazione standard tra membri dell'ensemble, globale e durante running; proxy epistemico di disaccordo. È ottenuta da <code>predict_all</code> e non dalla normalizzazione esterna usata da <code>predict</code> .
<code>spearman_returns</code>	Correlazione per ranghi tra $R^{true}(\tau)$ e $\hat{R}(\tau)$; misura la correttezza dell'ordinamento senza richiedere calibrazione di scala.
<code>spearman_returns_defined</code>	Indicatore 0/1; vale zero quando la correlazione non è definita, per esempio con return costanti.
<code>return_scatter</code>	Immagine W&B con return predetto sull'asse x e vero sull'asse y ; mostra ordinamento, calibrazione, outlier e compressione della scala.

Le metriche `reward/outcome/<esito>/*` sono emesse dopo l'update del reward. `count` è sempre registrato; `raw_return`, `norm_return`, `disc_return`, `mean_step_reward`, `terminal_reward` e `length` sono medie sulle traiettorie del relativo esito e sono omesse quando il conteggio è zero.

Per il replay buffer, posto $\Delta = r^{new} - r^{old}$, vengono registrati `sample_size`, medie e deviazioni standard dei reward vecchi/correnti, `delta_mean`, `delta_std`, `delta_abs_mean`, il 95-esimo percentile `delta_abs_p95`, `staleness_ratio` = $\text{mean}|\Delta|/\text{std}(r^{new})$ (con denominatore 1 se quasi nullo), la frazione `sign_flip_frac`, gli indicatori `relabel_enabled` e `critic_uses_current_reward`, e la correlazione `stored_current_corr` quando entrambe le deviazioni sono non nulle.

7.4 Tempi e assi di logging

`iterations` identifica l'iterazione esterna; `agent/time/total_timesteps` identifica il progresso dell'agente. Sono inoltre registrati `time/sample_rollout`, `time/train_reward_model`, `time/train_agent`, `time/imitation_diagnostics`, `time/loggings` e `time/total`. Servono a localizzare colli di bottiglia e a separare il costo delle diagnostiche dal training effettivo.

8 Checkpoint

Algorithm 20 SAVECHECKPOINT

Input: directory base D , iterazione i

- 1: crea $D/\text{checkpoint_format}(i, 4)$
 - 2: salva lo stato del reward model in `reward_model.pt`
 - 3: salva iterazione, loss, temperatura, flag di relabeling/normalizzazione e stati degli ottimizzatori in `reward_training.pt`
 - 4: salva l'agente
 - 5: **if** l'agente supporta il salvataggio del replay buffer **then**
 - 6: salva il buffer in `replay_buffer.pkl`
 - 7: **end if**
-

Il checkpoint corrente non memorizza esplicitamente `fragment_length`, batch size o learning rate: per una riproduzione completa tali valori devono essere conservati anche nella configurazione del run.